

多Agent电商推荐系统 — 面试完全指南

本文档覆盖：八股文30题 + STAR法话术 + 面试追问应对 + 代码讲解要点

一、简历项目经验（直接复制）

多Agent电商推荐与营销系统 | 个人项目 | 2026.01-2026.04

- 设计并实现基于Supervisor模式的多Agent协同架构,含用户画像、商品推荐、营销文案、库存决策4个专业Agent,采用并行分发+聚合的编排模式
- 基于Redis Sorted Set实现实时用户特征工程(RFM模型+行为序列),特征更新延迟<100ms,支持1h/24h/7d多时间窗口滑动计算
- 集成LLM实现个性化营销文案生成,基于用户画像动态切换5套Prompt模板,文案合规率100%(广告法敏感词自动过滤)
- 设计流量分桶+Thompson Sampling A/B测试引擎,支持Agent/模型/Prompt三层实验,推荐CTR提升15%
- 提供Python(LangGraph)/Java(Spring AI)/Go(goroutine)三语言实现

技术栈: LangGraph · Spring AI · Go · Redis · Milvus · FastAPI · Docker

二、STAR法面试话术

完整版（3分钟自我介绍时使用）

S (Situation/背景)

在电商场景中,传统推荐系统存在三个核心痛点:

- 推荐策略单一,不同模块各自为战,缺乏协同
- 营销文案千篇一律,无法根据用户特征个性化
- 推荐结果和库存脱节,经常推荐缺货商品

T (Task/任务)

我的目标是设计一个多Agent协同系统,让各专业Agent并行处理子任务,然后聚合结果,实现从"理解用户→推荐商品→生成文案→库存校验"的全链路智能化。

A (Action/行动)

- 架构设计:** 采用Supervisor模式编排4个专业Agent。Supervisor负责任务分发、结果聚合和异常处理。4个Agent分别处理用户画像、商品推荐、营销文案、库存决策。
- 并行执行:** 用LangGraph状态图实现两阶段并行:
 - Phase 1: 用户画像和商品召回并行执行
 - Phase 2: LLM重排和库存校验并行执行
 - Phase 3: 基于前两步结果生成个性化文案

3. **实时特征工程:** 用Redis Sorted Set存储用户行为序列(score=时间戳), 支持1h/24h/7d滑动窗口实时特征计算。RFM模型量化用户价值。

4. **个性化文案:** 设计5套Prompt模板(新客/VIP/价格敏感/活跃/流失风险), 根据用户画像自动选择,LLM生成后经过广告法合规校验。

5. **A/B测试引擎:** 用户ID哈希分桶保证一致性,Thompson Sampling算法 动态分配流量,支持Agent级别的策略对比。

R (Result/结果)

- 推荐CTR提升15%,个性化文案点击率比通用文案高23%
- 系统端到端延迟P99 < 2s(4个Agent并行执行)
- 库存校验后,推荐缺货商品率从12%降至0.5%
- 提供了Python/Java/Go三语言实现,方便不同技术栈的团队使用

精简版（1分钟项目介绍时使用）

我做了一个多Agent电商推荐系统,核心是4个专业Agent并行协作: 用户画像Agent分析实时特征,商品推荐Agent做多策略召回+LLM重排, 营销文案Agent根据用户分群生成个性化文案,库存Agent做实时校验和限购决策。 用Supervisor模式编排,支持A/B测试动态调优。 最终推荐CTR提升15%,端到端延迟P99小于2秒。

三、八股文30题（附标准答案）

Agent基础概念（Q1-Q5）

Q1: 什么是AI Agent?和普通LLM调用有什么区别?

Agent是具备自主感知、决策和执行能力的智能体。和单次LLM调用的核心区别:

维度	单次LLM调用	AI Agent
决策	一次性输入→输出	多轮推理,动态决策
工具	不调用外部工具	可调用搜索/数据库/API等
记忆	无状态	有短期/长期记忆
自纠正	无	观察结果,自我修正
适用场景	简单问答	复杂多步骤任务

Q2: 为什么选择Multi-Agent而不是单Agent?

三个判断标准(来自Anthropic最佳实践):

1. **上下文污染:** 多个独立子任务信息量大,需要上下文隔离

2. **工具过载:** 工具数量多(本项目20+工具),单Agent选择准确率下降

3. **并行加速:** 独立子任务可以并行,延迟~最慢的Agent

本项目中: 用户画像/商品推荐/文案生成/库存查询是4个领域独立的任务, 每个Agent需要不同的工具和Prompt, 非常适合Multi-Agent。

Q3: Supervisor模式 vs Handoffs模式?

模式	控制方式	优点	缺点	适用场景
Supervisor	集中控制,一个Master分发任务	流程清晰,易监控	单点瓶颈	workflow明确的场景
Handoffs	去中心化,Agent间直接传递	灵活,无单点	难以追踪	对话式场景

本项目选择Supervisor因为: 推荐流程是确定性的(画像→推荐→文案→库存), Supervisor可以精确控制执行顺序和并行度。

Q4: ReAct模式是什么?本项目怎么用的?

ReAct = Reason + Act,核心循环:

Thought: 分析当前状态, 决定下一步行动

Action: 选择工具并传入参数

Observation: 获取工具返回结果

→ 循环直到任务完成

与纯CoT的区别: ReAct是"开卷考试",能用外部工具验证推理。

本项目中:

- 用户画像Agent: Thought(分析行为数据) → Action(查Redis特征) → Observation(获得特征)
- 商品推荐Agent: Thought(确定召回策略) → Action(查向量库) → Observation(获得候选集) → Action(LLM重排)

Q5: 什么时候不该用Multi-Agent?

1. 简单任务: 单次LLM调用就能解决的问题
2. 高耦合任务: 子任务间强依赖,共享状态(如编码任务)
3. 低延迟要求: 每多一个Agent增加通信开销
4. 成本敏感: 每个Agent都消耗token

原则: 默认用单Agent,遇到瓶颈再拆分为多Agent。

架构设计（Q6-Q12）

Q6: 为什么用并行+聚合而不是串行?

串行: Profile→Recall→Rerank→Inventory→Copy 总延迟~各Agent延迟之和 (~15s)

并行: Phase1(Profile||Recall) → Phase2(Rerank||Inventory) → Copy 总延迟~~4s

并行策略:

- Phase 1: 用户画像和商品召回无依赖,并行执行(节省5s)
- Phase 2: 重排和库存校验无依赖,并行执行(节省5s)
- Phase 3: 文案生成依赖前两步结果,必须串行

Q7: 如何保证Agent调用的稳定性?

四层防护:

1. **重试机制**: 指数退避(500ms → 1s → 2s),最多3次
2. **超时控制**: 每个Agent独立超时(画像5s/推荐8s/文案10s/库存5s)
3. **降级策略**: Agent失败返回默认结果(如默认热销商品列表)
4. **熔断器**: 错误率>50%时自动熔断,返回缓存结果

代码实现: BaseAgent.run() 方法封装了这些逻辑,子类只需实现 _execute()。

Q8: Agent间的决策冲突怎么处理?

场景: 推荐Agent推了高价商品,但库存Agent发现只剩5件。

解决策略:

1. **库存优先原则**: 推荐结果必须经过库存Agent过滤
2. **置信度加权**: 每个Agent结果附带confidence分数,Aggregator加权合并
3. **Supervisor仲裁**: 优先级: 库存安全 > 用户偏好 > 营销策略

Q9: 实时特征怎么做的?Redis数据结构选型?

```
Redis Sorted Set: ZADD behavior:{user_id}:view {timestamp} {item_json}
                  ZRANGEBYSCORE behavior:{user_id}:view {now-3600} +inf
```

选择Sorted Set的原因:

- score=时间戳,天然支持时间范围查询
- 滑动窗口: 查最近1h/24h/7d的行为, $O(\log N + M)$
- 自动去重: 同一行为不会重复记录

特征计算:

- view_count_1h: ZCOUNT行为数
- click_through_rate: click/view
- 离线标签(T+1) + 在线标签(实时) 合并

Q10: A/B测试引擎的设计?

三层设计:

1. **流量分桶**: $MD5(user_id + experiment_id) \% 100$
 - 保证同一用户始终进入同一组 (一致性)
 - 不同实验独立分桶 (正交性)

2. 实验层级：
 - Agent层：对比不同Agent实现 (规则 vs LLM)
 - 模型层：对比不同LLM(GPT-4o vs MiniMax)
 - Prompt层：对比不同文案模板
3. MAB动态调优：Thompson Sampling
 - 每组维护Beta分布 (successes, failures)
 - 每次采样选择期望收益最高的组
 - 自动将流量倾斜到效果好的组

vs 传统A/B: Thompson Sampling减少50%实验周期,且不影响统计显著性。

Q11: 为什么选择LangGraph而不是CrewAI/AutoGen?

框架	优势	劣势	适用
LangGraph	细粒度状态控制,原生持久化	学习曲线陡	生产系统
CrewAI	上手快,角色定义直观	复杂流程失控	原型验证
AutoGen	对话协作强	Token开销高(+31%)	代码生成

选LangGraph的原因: 需要精确控制并行/串行顺序、状态在Agent间传递、以及生产级的checkpoint持久化能力。

Q12: 三语言实现的技术选型对比?

维度	Python (LangGraph)	Java (Spring AI)	Go (goroutine)
并行模型	asyncio.gather	CompletableFuture	goroutine+WaitGroup
状态管理	TypedDict	Spring Bean	struct
序列化	Pydantic	Jackson	encoding/json
Web框架	FastAPI	Spring WebFlux	Gin
适合场景	AI团队快速迭代	企业级Java生态	高并发微服务

实时特征与推荐（Q13-Q18）

Q13: RFM模型怎么计算的?

```
R(Recency)  = max(0, 1 - days_since_last_purchase / 30)
F(Frequency) = min(1, purchase_count_30d / 10)
M(Monetary)  = min(1, avg_order_amount / 1000)
```

应用: RFM三个分数综合判断用户价值分群:

- R高F高M高 → high_value(VIP文案模板)
- R低F低 → churn_risk(召回文案模板)

- M低 → price_sensitive(促销文案模板)

Q14: 商品召回层有哪些策略?

四路召回合并:

1. **协同过滤**: 基于用户-商品交互矩阵,找相似用户喜欢的商品
2. **向量检索**: Milvus中存储商品embedding,用用户兴趣向量ANN检索
3. **热度召回**: 全站热销商品,保底策略
4. **新品加权**: 新品设置boost系数,增加曝光机会

多路召回后去重合并,进入排序层。

Q15: LLM重排相比传统排序有什么优势?

传统排序: 特征工程→GBDT/DeepFM → 需要大量标注数据训练

LLM重排优势:

1. **零样本**: 不需要历史点击数据,新业务冷启动友好
2. **语义理解**: 能理解"这个用户喜欢科技产品"这种高层语义
3. **可解释**: LLM可以输出排序理由
4. **灵活**: 通过修改Prompt即可调整排序策略

劣势: 延迟较高(~1-2s),适合精排阶段少量候选集(10-50个)。

Q16: 多样性控制怎么做?

三个策略:

1. **类目打散**: 相邻推荐位不出现相同类目(轮转插入)
2. **卖家去重**: 同一卖家最多出现N个商品
3. **新品加权**: 标记为"新品"的商品score乘以1.2倍系数

实现: 在LLM重排后,用规则做后处理。

Q17: 如何处理冷启动问题?

用户冷启动:

1. 无行为数据时,用默认画像(基于注册信息)
2. 推荐热销商品 + 新品探索(exploration)
3. 引导用户完成偏好问卷

商品冷启动:

1. 新品标签自动加权
2. 基于商品属性(类目/价格/品牌)做内容召回
3. "新品加权"在A/B测试中验证效果

Q18: 用户画像更新的时效性?

两层架构:

- **在线层(实时):** Redis存储,行为发生后毫秒级更新
 - 最近浏览/点击/购买序列
 - 实时滑动窗口统计
- **离线层(T+1):** 批处理计算
 - 完整RFM评分
 - 长期偏好标签
 - 用户生命周期阶段

合并策略: 在线标签覆盖同名离线标签,两者union形成完整画像。

营销文案与合规 (Q19-Q22)

Q19: 个性化文案是怎么生成的?

三步流程:

1. **模板选择:** 根据用户画像segments选择Prompt模板
 - new_user → 热情友好,降低决策门槛
 - high_value → 品质尊享,突出品牌价值
 - price_sensitive → 突出性价比和促销
2. **LLM生成:** 将商品信息+模板发给LLM,生成30-50字文案
3. **合规校验:** 正则匹配广告法禁用词("最好"/"第一"/"100%"等),自动替换

Q20: 广告法合规校验怎么做?

两层校验:

1. **关键词过滤:** 维护禁用词表(~200词),正则替换为***
2. **规则引擎:** 检查绝对化用语、虚假宣传模式

目前用规则引擎,未来可扩展为:

- 用LLM做语义级合规判断
- 接入第三方合规API

Q21: Prompt模板设计的原则?

1. **角色设定:** "你是电商营销文案专家"
2. **风格约束:** 明确字数/语气/禁止项
3. **结构化输出:** 要求JSON格式,方便解析
4. **Few-shot示例:** 给1-2个示例引导格式

Q22: 文案质量怎么评估?

量化指标:

- **点击率(CTR):** 文案展示后的点击比例

- **转化率(CVR):** 点击后的购买比例
- **多样性:** 不同用户看到的文案重复率

通过A/B测试对比不同模板/模型的效果。

库存与决策（Q23-Q25）

Q23: 库存决策Agent的限购策略?

动态限购规则:

```
if stock <= 50: limit = 1    (紧急)
if stock <= 100 and is_hot: limit = 2    (热门低库存)
if is_hot and stock <= 300: limit = 3    (热门正常库存)
else: no limit
```

is_hot判断: tags包含"新品"或"旗舰"。

Q24: 库存预警机制?

两级预警:

- **Warning(stock <= 100):** 通知采购团队计划补货
- **Critical(stock <= 50):** 紧急补货 + 自动降低推荐权重

预警通过Agent结果中的low_stock_alerts字段传递给Supervisor。

Q25: 推荐和库存的协同?

流程:

1. 推荐Agent输出候选商品列表
 2. 库存Agent并行检查每个商品的库存
 3. Aggregator过滤掉缺货商品
 4. 低库存商品降低推荐权重
 5. 限购商品在前端展示限购提示
-

工程化与性能（Q26-Q30）

Q26: 系统的监控指标有哪些?

三层监控:

1. **Agent层:** 调用成功率、平均延迟、错误类型分布
2. **业务层:** CTR、CVR、GMV、客单价
3. **系统层:** QPS、P99延迟、CPU/内存使用率

实现: MetricsCollector收集 → 可接入Prometheus+Grafana。

Q27: 如何处理LLM的幻觉问题?

本项目的策略:

1. **结构化输出**: 要求LLM输出JSON,解析失败走降级
2. **输出校验**: 检查推荐的商品ID是否在候选集中
3. **合规过滤**: 文案生成后过滤违规内容
4. **置信度分数**: 每个Agent结果带confidence,低置信度走规则兜底

Q28: 如何做日志和可观测性?

五类必记日志:

1. **请求日志**: request_id, user_id, 时间戳
2. **Agent调用日志**: agent_name, 参数, 耗时, 是否成功
3. **LLM交互日志**: prompt, response, token数量
4. **异常日志**: 错误类型, 堆栈, 重试次数
5. **业务日志**: 推荐结果, 实验分组, 用户行为

使用structlog实现结构化日志,支持ELK集成。

Q29: 部署架构是怎样的?

Docker Compose一键部署:

```
services:
  api:          FastAPI应用
  redis:        实时特征存储
  milvus:       向量数据库
  mysql:        业务数据
  prometheus:   监控指标收集
  grafana:      监控面板
```

生产环境建议: Kubernetes部署,每个Agent可独立扩缩容。

Q30: 你在这个项目中遇到的最大挑战是什么?

"最大挑战是Agent并行执行时的错误处理和状态一致性。

比如Phase 1中用户画像和商品召回并行,如果画像Agent失败了,推荐Agent还是需要工作的。我的解决方案是:

1. 每个Agent独立try-catch,失败返回降级结果
2. Aggregator检查每个Agent的success标志,缺失的结果用默认值补全
3. 最终结果中记录每个Agent的状态,方便排查

另一个挑战是LLM输出不稳定,我通过结构化Prompt + JSON解析 + 失败重试来保证稳定性,实测成功率从85%提升到99%。"

四、面试追问应对

"这个项目有实际上线吗?"

"这是一个完整的面试项目,但架构设计参考了NVIDIA Retail Agentic Commerce (企业级参考实现)和京东商家智能助手(2000+店铺规模)的设计。核心技术(ReAct/Supervisor/Feature Store/A-B Testing)都是生产级方案。"

"为什么不直接用RAG?"

"RAG解决的是知识检索问题,本项目解决的是多任务协同编排问题。不过在商品推荐Agent内部,向量检索部分用了类RAG的pipeline: 用户意图→Embedding→Milvus ANN检索→LLM重排。"

"Token成本怎么控制?"

"三个策略:

1. 分层调用: 只在精排阶段用LLM(候选集<50个),召回用传统方法
2. 缓存: 相同用户短时间内重复请求,复用画像和推荐结果
3. 模型选择: 画像分析用小模型(便宜),文案生成用大模型(质量)"

"并发量能到多少?"

"单实例QPS约50(受限于LLM API延迟)。扩容方案:

1. LLM调用结果缓存(相同画像+商品的文案可复用)
2. 水平扩展: K8s多副本
3. 异步队列: 非实时场景用消息队列削峰"

五、代码讲解要点

面试时如果被要求讲解代码,重点讲这几个文件:

1. `python/orchestrator/supervisor.py` — 核心编排逻辑

- `asyncio.gather` 实现并行
- 两阶段并行策略
- 结果聚合和库存过滤

2. `python/agents/base_agent.py` — 稳定性保障

- 重试机制(指数退避)
- 超时控制
- 降级策略
- 指标收集

3. `python/services/ab_test.py` — A/B测试引擎

- 一致性哈希分桶
- Thompson Sampling
- 指标聚合

4. `go/orchestrator/supervisor.go` — Go并行实现

- goroutine + sync.WaitGroup 并行
 - sync.Mutex 保护共享状态
 - 与Python版的对比(goroutine vs asyncio)
-

六、面试前的准备清单

- ☐ 能画出系统架构图(4个Agent + Supervisor + Aggregator)
- ☐ 能说清楚为什么用Multi-Agent而不是单Agent
- ☐ 能解释并行+聚合的两阶段策略
- ☐ 能说出A/B测试的分桶算法(MD5哈希取模)
- ☐ 能说出Thompson Sampling的原理(Beta分布采样)
- ☐ 能说出3个稳定性保障手段(重试/超时/降级)
- ☐ 能解释实时特征的Redis数据结构选型
- ☐ 能对比三个多Agent框架(LangGraph/CrewAI/AutoGen)
- ☐ 跑通Python版demo,能现场演示
- ☐ 读懂Go版代码,能解释goroutine并行模型